كلية علوم وهندسة الحاسب
College of Computer Science & Engineering

جامعة جدة
University of Jeddah

CCCY-225 Software Security

Course Project

Part 1

Jana Zakai - 2310298

Nada Almutairi -2212349

1. System Overview

This car rental system determines the most suitable car based on:

1. Number of passengers
2. Number of rental days
3. Estimated trip mileage

The system then calculates the total trip cost for each car and selects the cheapest option, with comfort level used as a tie-breaker.

2. System pipeline

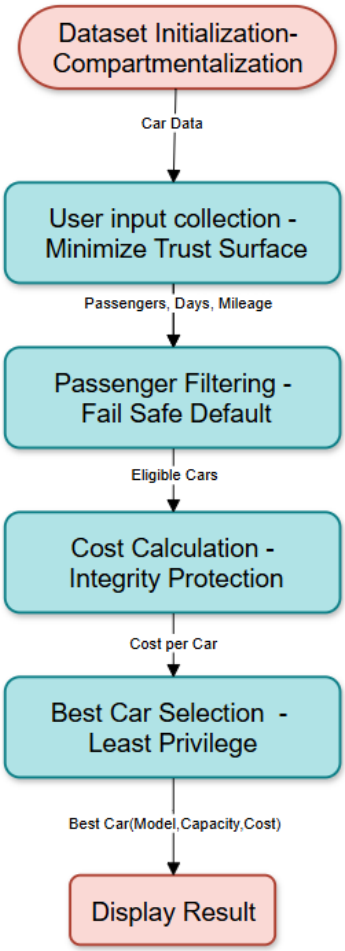

Figure 1- System pipeline with Applied Security Principles

The system operates in five main stages as presented in figure 1. In this section, each phase is explained in detail along with the security design principles applied at each specific phase.

2.1 Dataset Initialization

The program stores several cars inside a collection: ArrayList<Car>. Each car belongs to one of the project categories:

Category	Type	Max Passengers	Cost/day	MPG	Comfort	Make	Model
Coupe	Economy	4	45	35	Poor	Honda	Civic
Sedan	Intermediate	4	50	32	Medium	Toyota	Corolla
Hybrid	Intermediate	4	50	50	Medium	Toyota	Prius
SUV	Standard	5	55	28	Good	Toyota	RAV4
Crossover	Standard	5	55	27	Good	Mazda	CX5
Truck	Standard	5	55	22	Good	Ford	F150
Van/Minivan	Van	7	70	18	Medium	Chrysler	Pacifica
Sedan	Intermediate	4	50	19	Medium	dodge	charger
crossover	Standard	5	55	30	Good	kia	Soul

Table 1- list of cars

2.2.1 Security principle applied: Compartmentalization

Cars are represented as separate objects defined in the CarRepository class (figure 2). The main program interacts with them only through defined methods, specifically defined in the Car class (figure 3).

```
import java.util.ArrayList;

public class CarRepository {

    private ArrayList<Car> cars;

    public CarRepository() {

        cars = new ArrayList<>();

        cars.add(new Car(category: "Coupe", type: "Economy", maxPassengers: 4, costPerDay: 45, mpg:35, comfort:"Poor", make: "Honda", model: "Civic"));

        cars.add(new Car(category: "Sedan", type: "Intermediate", maxPassengers: 4, costPerDay: 50, mpg:32, comfort:"Medium", make: "Toyota", model: "Corolla"));

        cars.add(new Car(category: "Hybrid", type: "Intermediate", maxPassengers: 4, costPerDay: 50, mpg:50, comfort:"Medium", make: "Toyota", model: "Prius"));

        cars.add(new Car(category: "SUV", type: "Standard", maxPassengers: 5, costPerDay: 55, mpg:28, comfort:"Good", make: "Toyota", model: "RAV4"));

        cars.add(new Car(category: "Crossover", type: "Standard", maxPassengers: 5, costPerDay: 55, mpg:27, comfort:"Good", make: "Mazda", model: "CX5"));

        cars.add(new Car(category: "Truck", type: "Standard", maxPassengers: 5, costPerDay: 55, mpg:22, comfort:"Good", make: "Ford", model: "F150"));

        cars.add(new Car(category: "Van/Minivan", type: "Van", maxPassengers: 7, costPerDay: 70, mpg:18, comfort:"Medium", make: "Chrysler", model: "Pacifica"));
    }

    public ArrayList<Car> getCars() {
        return cars;
    }
}
```

Figure 2 - Cars information defined in the CarRepository class

```java
    public Car(String category, String type, int maxPassengers,
               double costPerDay, double mpg, String comfort,
               String make, String model) {

        this.category = category;
        this.type = type;
        this.maxPassengers = maxPassengers;
        this.costPerDay = costPerDay;
        this.mpg = mpg;
        this.comfort = comfort;
        this.make = make;
        this.model = model;
    }

    public boolean fitsPassengers(int passengers) {
        return passengers <= maxPassengers;
    }

    public double calculateTripCost(int days, double mileage) {

        double rentalCost = days * costPerDay;
        double fuelCost = (mileage / mpg) * 2.25;

        return rentalCost + fuelCost;
    }

    public int getComfortScore() {

        if (comfort.equalsIgnoreCase(anotherString: "Good")) return 3;
        if (comfort.equalsIgnoreCase(anotherString: "Medium")) return 2;
        return 1;
    }

    public String getCarInfo() {
        return make + " " + model + " (" + category + ")";
    }
}
```

Figure 3- car class

2.2 User Input Collection

The program asks the user for number of passengers, rental days, and estimated trip mileage.

2.2.1 Security principle applied: Minimize Trust Surface

User input is never trusted directly. Validation checks ensure that:

- passengers > 0
- days > 0
- mileage > 0
- passengers ≤ 7

If input is invalid, the system stops safely.

```
if(passengers <=0 || passengers >7 ||
   days <=0 || mileage <=0){

    System.out.println(x: "Invalid input.");
    return;
}
```

Figure 4 input checker in CarRentalApp class

2.3 Passenger Filtering

The system filters cars through a three-phased algorithm defined in the findBestCar method under the Car Selector Class (figure 5).

```java
public class CarSelector {

    public Car findBestCar(ArrayList<Car> cars, int passengers, int days, double mileage) {

        double bestCost = Double.MAX_VALUE;
        Car bestCar = null;

        for(Car car : cars){

            if(!car.fitsPassengers(passengers))
                continue;

            double cost = car.calculateTripCost(days, mileage);

            if(cost < bestCost){
                bestCost = cost;
                bestCar = car;
            }

            else if(cost == bestCost &&
                    car.getComfortScore() > bestCar.getComfortScore()){

                bestCar = car;
            }
        }

        return bestCar;
    }
}
```

Figure 5- full filtering algorithm defined in the FindBestCar method

First, the system filters cars based on the number of passengers (figure 6), selecting the cars that can fit the passengers. The code follows the logic described below:

FOR each car in carList
IF passengers <= car.maxPassengers
keep the car
ELSE
discard the car

```java
public Car findBestCar(ArrayList<Car> cars, int passengers, int days, double mileage) {

    double bestCost = Double.MAX_VALUE;
    Car bestCar = null;

    for(Car car : cars){

        if(!car.fitsPassengers(passengers))
            continue;
```

Figure 6- Part of method in CarSelector class for filtering passengers

2.3.1 Security principle applied: Fail-Safe Default

Cars are rejected unless explicitly allowed.

2.4 Cost Calculation

The second filtering phase of the algorithm is calculating the car cost, as shown in figure 7, which uses the calcuateTripCost method defined in the Car Class (figure 8). The method includes computing the fuel cost, rental cost, and total cost for every remaining car. The equations used are defined below.

*fuelCost = (tripMileage / mpg) * 2.25*

*rentalCost = days * costPerDay*

totalCost = rentalCost + fuelCost

```
double cost = car.calculateTripCost(days, mileage);
```

Figure 7- Part of method in CarSelector class for cost calculation

```
public double calculateTripCost(int days, double mileage) {

    double rentalCost = days * costPerDay;
    double fuelCost = (mileage / mpg) * 2.25;

    return rentalCost + fuelCost;
}
```

Figure 8- calculateTripCost method defined in Car class

2.5 Best Car Selection

The last phase of the filtering process, shown in figure 9, is selecting the car with lowest total cost. If multiple cars have the same cost, the system chooses the car with higher comfort level.

```
    if(cost < bestCost){
        bestCost = cost;
        bestCar = car;
    }

    else if(cost == bestCost &&
            car.getComfortScore() > bestCar.getComfortScore()){

        bestCar = car;
    }
}

return bestCar;
```

Figure 9- Part of method in CarSelector class for best car selection

2.6 Output

The system prints Car Make, Car Model, Passenger Capacity and Total Trip Cost.

```
Number of passengers: 5
Number of rental days: 2
Trip mileage: 44
-----------------------------------------------

Best Car: Toyota RAV4 (SUV)
Max Passengers: 5
Total Cost: $113.54
```

Figure 10- sample output

3. Security Requirements Applied

3.1 Integrity

Integrity ensures that system data remains accurate and is not modified in an unauthorized manner.

In this application, integrity is enforced by protecting all vehicle-related data such as mileage (MPG), rental cost, and passenger capacity from external modification. This is achieved through encapsulation, where all attributes in the Car class are declared as private and accessed only through controlled methods.

Additionally, the system does not provide setter methods for critical attributes, ensuring that once a car object is created, its data cannot be altered. All computations, such as total trip cost, are performed through dedicated methods to avoid inconsistent or incorrect calculations.

```
14      private double costPerDay;
15      private double mpg;
16      private String comfort;
17      private String make;
18      private String model;
19
20      public Car(String category, String type, int maxPassengers,
21              double costPerDay, double mpg, String comfort,
22              String make, String model) {
23
24          this.category = category;
25          this.type = type;
26          this.maxPassengers = maxPassengers;
27          this.costPerDay = costPerDay;
28          this.mpg = mpg;
29          this.comfort = comfort;
30          this.make = make;
31          this.model = model;
32      }
33
34      public boolean fitsPassengers(int passengers) {
35          return passengers <= maxPassengers;
36      }
37
38      public double calculateTripCost(int days, double mileage) {
39
40          double rentalCost = days * costPerDay;
41          double fuelCost = (mileage / mpg) * 2.25;
42
43          return rentalCost + fuelCost;
44      }
```

Figure 11- data integrity implementation in program

Figure 11 shows how data integrity is maintained using encapsulation and controlled cost calculation inside the Car class.

3.2 Availability

Availability ensures that the system remains operational and accessible under normal conditions without failure.

In this application, availability is achieved through strict input validation and proper handling of edge cases. The system verifies that user inputs such as number of passengers, rental days, and mileage are within valid ranges before proceeding with computations. This prevents runtime errors and system crashes.

Furthermore, the system gracefully handles cases where no suitable car is found by providing appropriate user feedback instead of terminating unexpectedly.

```
26
27          if(passengers <=0 || passengers >7 ||
28             days <=0 || mileage <=0){
29
30              System.out.println(x: "Invalid input.");
31              return;
32          }
33
34          CarRepository repo = new CarRepository();
35          CarSelector selector = new CarSelector();
36
37          Car bestCar = selector.findBestCar(repo.getCars(), passengers, days, mileage);
38
39          if(bestCar != null){
40              System.out.println(x: "\nBest Car:");
41              System.out.println(bestCar.getCarInfo());
42          }
43          else{
44              System.out.println(x: "No suitable car found.");
45          }
46
47          scanner.close();
48      }
49  }
```

Figure 12- data availability implementation in program

Figure 12 demonstrates input validation and safe handling of invalid or edge cases to maintain system availability.

3.3 Authenticity

Authenticity ensures that the data being processed by the system is valid and originates from a legitimate source.

In this application, authenticity is enforced by validating all user inputs before they are used in any computation. The system ensures that input values are within realistic and acceptable bounds and that the correct data types are provided.

This prevents invalid, corrupted, or malicious input from affecting system behavior or producing incorrect results.

```
8   public class CarRentalApp {
9
10
        Run main | Debug main | Run | Debug
11      public static void main(String[] args) {
12
13          Scanner scanner = new Scanner(System.in);
14
15          System.out.print(s: "Number of passengers: ");
16          int passengers = scanner.nextInt();
17
18          System.out.print(s: "Number of rental days: ");
19          int days = scanner.nextInt();
20
21          System.out.print(s: "Trip mileage: ");
22          double mileage = scanner.nextDouble();
23
24          if(passengers <=0 || passengers >7 ||
25             days <=0 || mileage <=0){
26
27              System.out.println(x: "Invalid input.");
28              return;
29          }
```

Figure 13- data authenticity implementation in program

Figure 13 shows how user input is collected and validated to ensure authenticity of the data.

4. Contribution

name	Jana Zakai	Nada Almutairi
contributions	Added car information with the following category types: coupe, sedan, hybrid, crossover	Added car information with the following category types: SUV, Van, Crossover, Truck
	Designed filtering algorthim	Designed system pipeline
	Defined security principles applied in program	Defined security requirements applied in program

Table 2- contributions to project

Table 2 describes the specific tasks and contributions to the project by each member.

5. Github repository

Below is the link to the repository containing the project's source code.

https://github.com/Jzakai/CarRentalApp/tree/master